

Dynamic Linear Models and the Kalman Filter

David Puelz

The University of Austin

The Problem

The Model

The Kalman Filter

The Prior

Implementation

Tuning

Summary



Noisy measurements of a moving target

We want to track something that changes over time, but our measurements have error.

Examples:

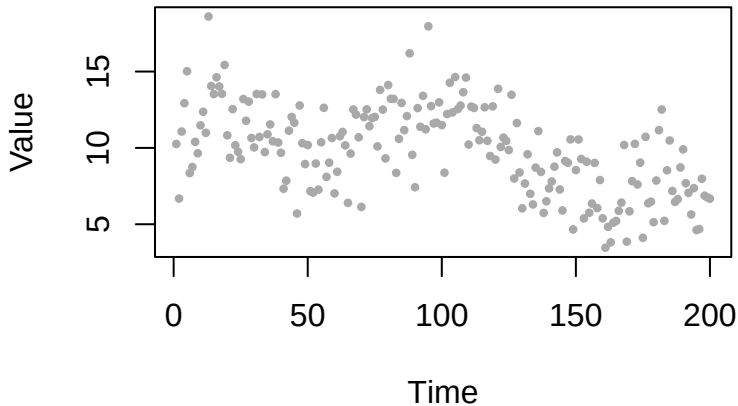
- True unemployment rate (surveys have sampling error)
- Patient's blood pressure (instruments have noise)
- Demand for a product (observed sales are noisy)

Goal: Estimate the true underlying value from noisy data.

Raw data



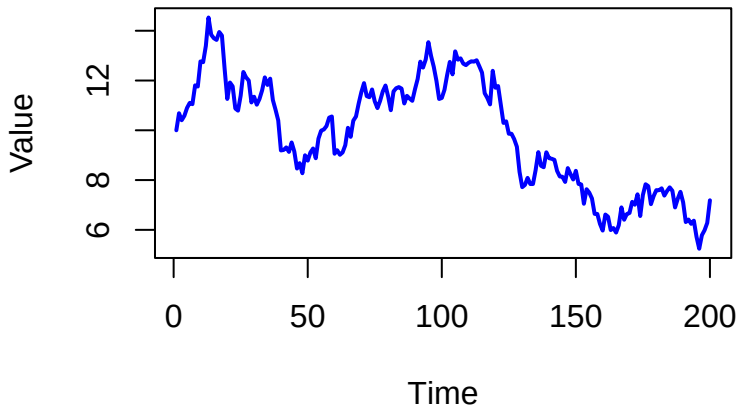
What we observe: noisy measurements over time (gray points).





The true signal

What we *don't* observe: the underlying value θ_t (blue line).



The Problem

The Model

The Kalman Filter

The Prior

Implementation

Tuning

Summary



The model

Let θ_t = true value, Y_t = observation

Observation equation:

$$Y_t = \theta_t + v_t, \quad v_t \sim N(0, V)$$

We see the truth plus measurement noise.

State equation:

$$\theta_t = \theta_{t-1} + w_t, \quad w_t \sim N(0, W)$$

The truth evolves as a random walk.



The two variances

- V = observation noise variance
 - Larger $V \rightarrow$ noisier measurements
- W = state evolution variance
 - Larger $W \rightarrow$ truth can change more

The ratio W/V will determine how the filter behaves.



What we're estimating

The quantity of interest: θ_t = the true underlying value at time t .

- We *observe* Y_t (noisy).
- We *do not observe* θ_t (the signal).
- **Goal:** Estimate θ_t at each time t from the observations $Y_1, \dots, Y_t$.

The Kalman filter gives us an optimal estimate of θ_t (and its uncertainty) at every t .

The Problem

The Model

The Kalman Filter

The Prior

Implementation

Tuning

Summary



The idea

At each time t , combine two pieces of information:

1. **Prediction:** Where we think θ_t is (before seeing Y_t)
2. **Observation:** The new data point Y_t

The filter computes a **weighted average**:

$$\text{Estimate} = (1 - K) \times \text{Prediction} + K \times \text{Observation}$$

where K is the **Kalman gain** (between 0 and 1).



The conditional distributions we care about

At each time t , we care about three distributions:

- **Prior:** $p(\theta_t \mid Y_1, \dots, Y_{t-1})$ (belief about θ_t *before* seeing Y_t)
- **Likelihood:** $p(Y_t \mid \theta_t)$ (how the observation depends on the truth)
- **Target (posterior):** $p(\theta_t \mid Y_1, \dots, Y_t)$ (belief about θ_t *after* seeing Y_t)



How we get the target: Bayes' law

Bayes' law says:

$$p(\theta_t \mid Y_{1:t}) \propto p(\theta_t \mid Y_{1:t-1}) \times p(Y_t \mid \theta_t)$$

$$\text{posterior} \propto \text{prior} \times \text{likelihood}$$

So: combine our prediction (prior) with the new observation (likelihood) to get our updated belief (posterior).



Prior and likelihood from the model

Likelihood (observation equation):

$$Y_t \mid \theta_t \sim N(\theta_t, V)$$

Prior (state equation + last posterior):

$$\theta_t \mid Y_{1:t-1} \sim N(a_t, R_t), \quad a_t = m_{t-1}, \quad R_t = C_{t-1} + W$$

Bayes: posterior $\propto$ prior $\times$ likelihood $\Rightarrow p(\theta_t \mid Y_{1:t})$.



The algorithm

Step 1: PREDICT

$$a_t = m_{t-1} \quad \text{(predicted mean)}$$

$$R_t = C_{t-1} + W \quad \text{(predicted variance)}$$

Step 2: UPDATE

$$K_t = \frac{R_t}{R_t + V} \quad \text{(Kalman gain)}$$

$$m_t = a_t + K_t(Y_t - a_t) \quad \text{(updated mean)}$$

$$C_t = (1 - K_t)R_t \quad \text{(updated variance)}$$



Where do the prediction formulas come from?

Recall the state equation: $\theta_t = \theta_{t-1} + w_t$

Predicted mean:

- Our best guess for θ_{t-1} was m_{t-1}
- The expected change is $E[w_t] = 0$
- So our prediction is: $a_t = m_{t-1} + 0 = m_{t-1}$

Predicted variance:

- We were uncertain about θ_{t-1} by C_{t-1}
- The random walk adds uncertainty W
- Variances add: $R_t = C_{t-1} + W$



Where do the update formulas come from?

We have two pieces of information about θ_t :

- Prediction a_t with variance R_t
- Observation Y_t with variance V

Key idea: When combining estimates, weight by precision (1/variance).

Precision of prediction: $1/R_t$

Precision of observation: $1/V$

Total precision: $1/R_t + 1/V$



Deriving the Kalman gain

Weight on observation = (its precision) / (total precision):

$$K = \frac{1/V}{1/R_t + 1/V}$$

Multiply top and bottom by $R_t \cdot V$:

$$K = \frac{R_t}{R_t + V}$$

$$\text{Weight on prediction} = 1 - K = \frac{V}{R_t + V}$$



Deriving the updated mean and variance

Updated mean (weighted average):

$$m_t = (1 - K) \cdot a_t + K \cdot Y_t = a_t + K(Y_t - a_t)$$

Updated variance (combined precision):

$$\frac{1}{C_t} = \frac{1}{R_t} + \frac{1}{V}$$

Solving for C_t :

$$C_t = \frac{R_t \cdot V}{R_t + V} = R_t \cdot \frac{V}{R_t + V} = R_t(1 - K) = (1 - K)R_t$$



The Kalman gain

$$K_t = \frac{R_t}{R_t + V} = \frac{\text{prediction uncertainty}}{\text{total uncertainty}}$$

- $K \approx 1$: Trust the observation (prediction is uncertain)
- $K \approx 0$: Trust the prediction (observation is noisy)

The Problem

The Model

The Kalman Filter

The Prior

Implementation

Tuning

Summary



Starting the filter

The filter needs a **starting point**:

- m_0 = initial guess for the true value
- C_0 = how uncertain we are about that guess

This is called the **prior**—what we believe before seeing any data.



Choosing the prior

If you have prior knowledge:

- Set m_0 to your best guess
- Set C_0 to reflect your confidence

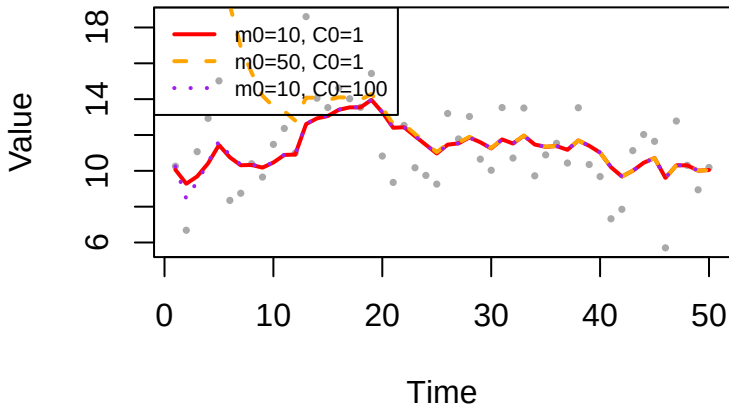
If you don't know:

- Set m_0 to first observation: $m_0 = Y_1$
- Set C_0 large (“diffuse prior”)—says “I’m not sure”

Good news: With enough data, the prior washes out.



Prior doesn't matter much (with enough data)



Different starting points converge to the same estimates (first 50 time steps shown).

The Problem

The Model

The Kalman Filter

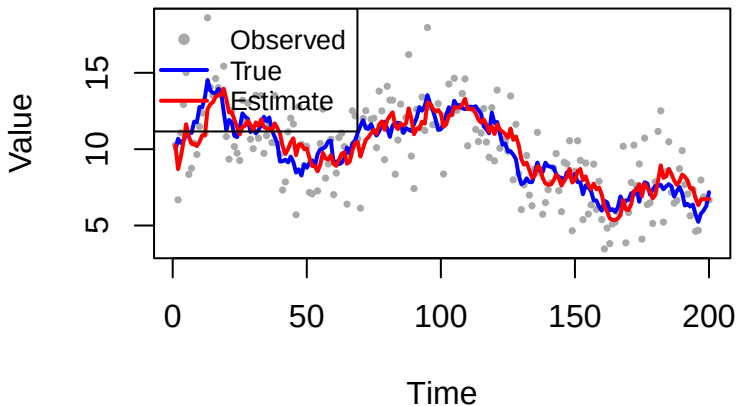
The Prior

Implementation

Tuning

Summary

```
kalman_filter <- function(Y, V, W, m0, C0) {  
  n <- length(Y)  
  m <- C <- numeric(n)  # storage for estimates  
  a <- m0                # initialize  
  
  for (t in 1:n) {  
    # PREDICT:  $R_t = C_{t-1} + W$  (prediction variance)  
    R <- (if (t > 1) C[t-1] else C0) + W  
  
    # UPDATE  
    K <- R / (R + V)  
    m[t] <- a + K * (Y[t] - a)  
    C[t] <- (1 - K) * R  
  
    a <- m[t]  # for next iteration  
  }  
  return(list(m = m, C = C))  
}
```



The filter recovers the true signal from noisy observations.



With uncertainty

```
Error:  '\%' is an unrecognized escape in character  
string (<text>:9:54)
```

The filter also tells us how uncertain we are.

The Problem

The Model

The Kalman Filter

The Prior

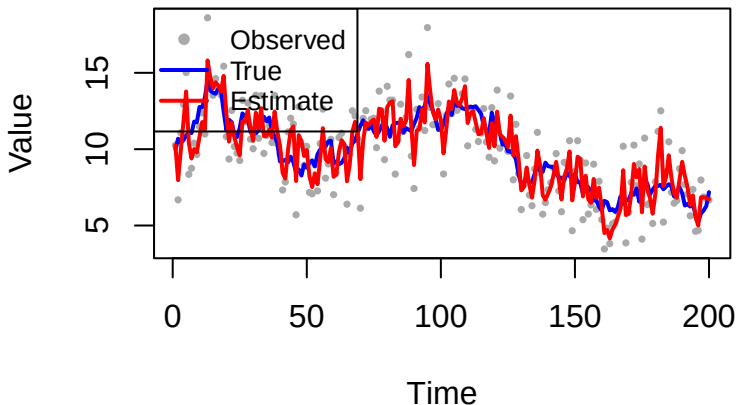
Implementation

Tuning

Summary



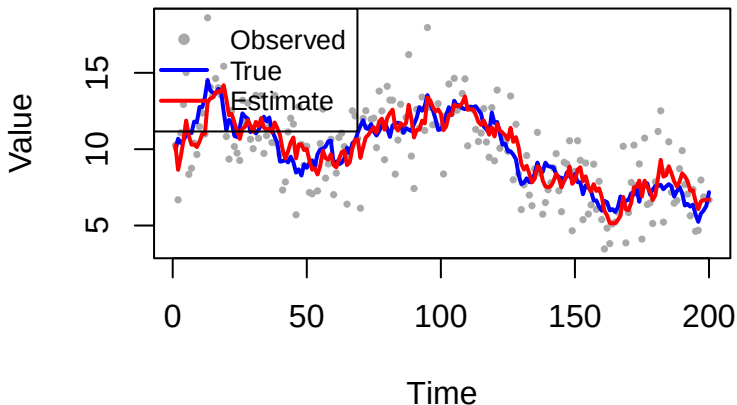
Little smoothing ($W/V = 1$)



Estimate tracks the data closely; little smoothing.



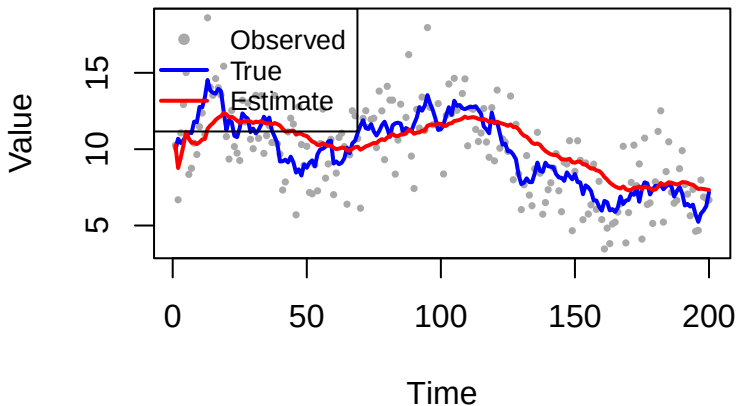
Some smoothing ($W/V = 0.1$)



Moderate smoothing; balance between noise and responsiveness.



Extreme smoothing ($W/V \approx 0.0025$)



Very

smooth estimate; slow to react to changes.

The Problem

The Model

The Kalman Filter

The Prior

Implementation

Tuning

Summary



Key points

1. **Model:** Truth is a random walk, observed with noise
2. **Algorithm:** PREDICT $\rightarrow$ UPDATE at each time step
3. **Kalman gain:** Weights prediction vs. observation
4. **Prior:** Starting guess (m_0, C_0) ; washes out with data
5. **Tuning:** W/V ratio controls smoothness



The Kalman filter in one sentence

Optimally combine what you predicted
with what you observed,
weighting by their uncertainties.

The Problem

The Model

The Kalman Filter

The Prior

Implementation

Tuning

Summary



Bayes' law (reminder)

Bayes' law tells us how to update beliefs in light of new data:

$$p(\theta \mid \text{data}) = \frac{p(\theta) p(\text{data} \mid \theta)}{p(\text{data})} \propto p(\theta) \times p(\text{data} \mid \theta)$$

- **Prior** $p(\theta)$: belief about θ *before* seeing the data
- **Likelihood** $p(\text{data} \mid \theta)$: how the data depend on θ
- **Posterior** $p(\theta \mid \text{data})$: belief about θ *after* seeing the data

So: **posterior** $\propto$ **prior** $\times$ **likelihood**.

The **Kalman filter** does exactly this at every time step: the PREDICT step gives the prior for θ_t , the observation Y_t provides the likelihood, and the UPDATE step gives the posterior.



The filter is Bayesian updating

At each time t , the Kalman filter does exactly **Bayesian inference**:

1. **Prior:** Belief about θ_t *before* seeing Y_t
→ from the PREDICT step: $\theta_t \mid Y_{1:t-1} \sim N(a_t, R_t)$
2. **Likelihood:** How the observation Y_t depends on θ_t
→ observation equation: $Y_t \mid \theta_t \sim N(\theta_t, V)$
3. **Posterior:** Belief about θ_t *after* seeing Y_t
→ from the UPDATE step: $\theta_t \mid Y_{1:t} \sim N(m_t, C_t)$

So: **prior** $\times$ **likelihood** $\propto$ **posterior**, at every time step.

Derivation: Prior and likelihood

Prior (from state equation and last posterior):

$$\theta_t \mid Y_{1:t-1} \sim N(a_t, R_t), \quad a_t = m_{t-1}, \quad R_t = C_{t-1} + W$$

So $p(\theta_t \mid Y_{1:t-1}) \propto \exp\left(-\frac{1}{2R_t}(\theta_t - a_t)^2\right)$.

Likelihood (from observation equation):

$$Y_t \mid \theta_t \sim N(\theta_t, V)$$

So $p(Y_t \mid \theta_t) \propto \exp\left(-\frac{1}{2V}(Y_t - \theta_t)^2\right)$.



Derivation: Posterior via Bayes

Fact: Product of two Gaussians (in θ) $\propto$ Gaussian.

Posterior $\propto$ prior $\times$ likelihood gives exponent quadratic in θ_t ;
completing the square yields:

- **Precision:** $\frac{1}{C_t} = \frac{1}{R_t} + \frac{1}{V}$

- **Mean (precision-weighted):**

$$m_t = \frac{(1/R_t) a_t + (1/V) Y_t}{1/R_t + 1/V} = a_t + K_t(Y_t - a_t)$$

Handout: *Posterior via Bayes: Product of Two Gaussians* has the full step-by-step derivation (expand, collect terms, read off C_t and m_t).



Summary: Bayesian view

- The Kalman filter is **recursive Bayesian inference**: at each t , we update from prior to posterior using the new observation.
- Because the prior and likelihood are **Gaussian**, the posterior is Gaussian and the update has a closed form (no MCMC needed).
- The Kalman gain $K_t = R_t / (R_t + V)$ is exactly the weight that Bayes' rule gives to the observation when combining with the prior.

This same idea (prior $\rightarrow$ likelihood $\rightarrow$ posterior) appears throughout Bayesian statistics; the Kalman filter is the dynamic, sequential version for time series.